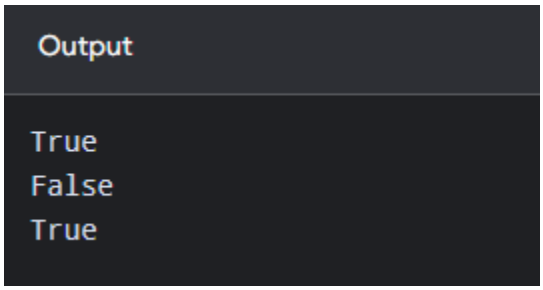
Propositional Logic

```
class Var:
    def __init__(self, name):
        self.name = name

class Op:
    def __init__(self, op, left, right=None):
        self.op = op
        self.left = left
        self.right = right

def get_vars(node):
    if isinstance(node, Var):
        return {node.name}
    elif isinstance(node, Op):
        if node.right:
            return get_vars(node.left) | get_vars(node.right)
        return get_vars(node.left)
    return set()

def pl_true(node, model):
    if isinstance(node, Var):
        return model[node.name]
    if node.op == 'NOT':
        return not pl_true(node.left, model)
    if node.op == 'AND':
        return pl_true(node.left, model) and pl_true(node.right, model)
    if node.op == 'OR':
        return pl_true(node.left, model) or pl_true(node.right, model)
    if node.op == 'IMPLIES':
        return not pl_true(node.left, model) or pl_true(node.right, model)

def tt_check_all(kb, alpha, symbols, model):
    if not symbols:
```

```
        if pl_true(kb, model):
            return pl_true(alpha, model)
        return True

    p = symbols.pop()
    model_t = model.copy()
    model_t[p] = True
    model_f = model.copy()
    model_f[p] = False
    symbols.append(p)

    return tt_check_all(kb, alpha, symbols[:-1], model_t) and tt_check_all(kb,
alpha, symbols[:-1], model_f)

def tt_entails(kb, alpha):
    symbols = list(get_vars(kb) | get_vars(alpha))
    return tt_check_all(kb, alpha, symbols, {})

Q = Var('Q')
P = Var('P')
R = Var('R')

kb = Op('AND', Op('IMPLIES', Q, P), Op('AND', Op('IMPLIES', P, Op('NOT',
Q)), Op('OR', Q, R)))
print(tt_entails(kb, R))
print(tt_entails(kb, Op('IMPLIES', R, P)))
print(tt_entails(kb, Op('IMPLIES', Q, R)))
```